
HACKATHON IT · ÉDITION 2026

L'Intelligence Artificielle au service de la sécurité financière

Détecter, prévenir et combattre la fraude

Épreuve individuelle · 4 heures · Python · Notation automatique par CI

Thème

Ce thème invite les participants à réfléchir à des solutions innovantes basées sur l'intelligence artificielle pour l'identification des transactions suspectes, la prévention des fraudes financières, l'amélioration de la sécurité des paiements et la protection des utilisateurs contre les risques liés à la cybercriminalité financière.

Format de l'épreuve

- **Durée** : 4 heures.
- **Participation** : en solo.
- **Langage** : Python.
- **Évaluation automatique** : une suite de tests s'exécute via la CI à chaque Pull Request. Le score apparaît dans l'onglet `Checks` de la PR (X/11 tests publics).

Objectif

Construire un détecteur de fraude en complétant la fonction `detect_fraud`. Pour chaque transaction, le programme attribue un score de risque, un verdict et une justification lisible. L'enjeu n'est pas seulement de repérer les fraudes évidentes, mais aussi de **ne pas signaler à tort** des transactions légitimes.

Dépôt et démarrage

Dépôt : github.com/INTELO2026/fraud-challenge

1. Forkez le dépôt `INTELO2026/fraud-challenge`.
2. Clonez votre fork :

```
git clone https://github.com/VOTRE-PSEUDO/fraud-challenge.git
```
3. Installez les dépendances :

```
pip install -r requirements.txt
```
4. Vérifiez que les tests se lancent (ils échouent au départ, c'est normal) :

```
pytest tests/ -v
```
5. Implémentez `detect_fraud` dans `fraud_detection.py`.
6. Poussez votre code et ouvrez une Pull Request vers `INTELO2026/fraud-challenge`.
7. Consultez votre score dans l'onglet `Checks` de la PR (X/11 tests publics).

La fonction à implémenter

Seule la fonction `detect_fraud` est obligatoire. La fonction `load_transactions` (lecture du CSV) vous est déjà fournie : ne perdez pas de temps sur le parsing.

```
| def detect_fraud(transactions):
```

Entrée : la liste complète des transactions (des dictionnaires). Vous recevez tout le lot d'un coup, ce qui vous permet de comparer chaque transaction à l'historique du client.

Sortie : une liste de résultats, un par transaction, dans le même ordre. Chaque résultat contient :

- `transaction_id` : le même identifiant que l'entrée.
- `fraud_score` : nombre entre 0.0 et 1.0.
- `is_suspicious` : booléen (True si la transaction doit être signalée).
- `reason` : courte explication lisible du verdict.

Format d'une transaction

Colonne	Type	Description
<code>transaction_id</code>	texte	Identifiant unique
<code>timestamp</code>	texte ISO 8601	Date et heure ; peut être vide
<code>user_id</code>	texte	Identifiant du client
<code>amount</code>	nombre	Montant ; peut être vide, nul ou négatif
<code>currency</code>	texte	Devise (EUR, USD, XOF...)
<code>merchant</code>	texte	Nom du commerçant
<code>country</code>	texte	Pays (code ISO, ex. FR) ; peut être vide
<code>card_present</code>	booléen	Carte physique présente ou non

Les données réelles sont imparfaites : champs manquants, doublons, montants aberrants, horodatages désordonnés. Votre programme ne doit jamais planter.

Les trois paliers de difficulté

Votre note correspond au nombre de tests réussis, du plus simple au plus exigeant. **Une partie des tests est cachée** et n'est exécutée qu'à l'évaluation finale : faire passer les tests publics ne garantit pas tous les points. Codez une vraie logique, pas des réponses en dur.

- **Niveau 1 — Fondamentaux** : la sortie respecte le format ; les anomalies évidentes (montant négatif ou nul, champ manquant) sont signalées.
- **Niveau 2 — Logique métier** : montant anormal par rapport à l'historique du client, fréquence de transactions suspecte, incohérence géographique (deux pays éloignés en trop peu de temps).
- **Niveau 3 — Finesse** : éviter les faux positifs (une transaction inhabituelle mais légitime ne doit pas être signalée) et gérer les cas limites.

Interface de démonstration (facultative, bonus)

Le site web n'est **pas obligatoire** : seule la fonction `detect_fraud` est notée par les tests. En bonus, vous pouvez créer une interface dans `app.py` (fonction `render_interface`) pour présenter vos résultats. Elle sera appréciée par le jury lors de la démo :

```
| streamlit run app.py
```

Décrivez votre interface dans le modèle de Pull Request.

Évaluation et classement

- **Pendant l'épreuve** : score visible dans l'onglet `Checks` de la PR (X/11 tests publics).
- **Classement final** : les tests cachés sont évalués par les organisateurs après le deadline.
- **En cas d'égalité** : la lisibilité du code, la pertinence des justifications et, le cas échéant, la qualité de l'interface de démo départagent.